Project Overview: PME Extensions

This document provides a high-level overview of the recent enhancements and changes introduced through the PME (Parquet Modular Encryption) Extensions project.

Key Changes and Enhancements

Feature Area	Description of Change/Extension
Granular Encryption	Encryption of individual columns is a traditional feature of PME, however all encrypted columns (for a given table) were required to use the same encryption algorithm. We have added more granular support where column-specific encryption algorithms are allowed. To achieve this, we have extended the mechanism to define per-table and per-column encryption properties from both Python and C++ applications.
External Encryptors	Originally, PME offered two sub-flavors of a single encryption algorithm (AES-GCM). These are internal to PME, with no easy possibility of adding other encryptors. We have extended PME with a more modular architecture which allows external encryptors to be user-defined. External encryptors are treated like plugins, and are integrated (i.e. loaded) via shared libraries (e.g. `*.so` files). This, for example, enables service-based encryption and new encryption algorithms to be used with PME.
Per-value encryption	Column encryption in PME traditionally occurred at the data-page level (a sub-division of the column), where the data of the page was treated as a single blob to be encrypted. We have created extensions where additional page encoding information is passed to the encryptors to allow them to perform a new per-value encryption, or to continue with the traditional page-level encryption.
Context-aware encryption	A context-aware encryptor can use the added context to customize the behavior of the encryptor, e.g. more fine-grained robust ACL, geo-location specifics, or simple hints for application specific optimizations.

Feature Area	Description of Change/Extension
	We enabled the passing of additional context from the application down to the external encryptor to pass down application-to-encryptor parameters, constituting a private contract between the two parts.
Encryptor-provided metadata	To allow for more flexibility for encryptors, we have created a new feature where encryptors can pass back additional metadata at page level, and have the metadata stored alongside the encrypted data. At decryption time, encryptors will receive the metadata originally passed at encryption time. This feature did not require modification of the Parquet format.
Variable-length encryption	Previously, PME assumed algorithms like AES-GCM could pre-compute ciphertext/plaintext sizes, so Arrow allocated a fixed output buffer before calling encrypt/decrypt. We created an extension where Arrow instead passes a resizable buffer to the external encryptor/decryptor, which resizes it during the operation and returns the actual number of bytes produced. This enables broader external encryptor compatibility (e.g., padding, framing, or provider-specific formats) while preserving the existing fixed-size path for AES-GCM and similar algorithms.